

EXPERIMENT NO.4

Name:Aryan Anil Patankar

Class:D20A

Roll No:38

Batch:B

Aim:Deploying a Voting/Ballot Smart Contract

Theory:-

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping:**

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal

they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage:**

In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory:**

In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type**

Converter help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

□ **Code:**

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    // This declares a new complex type which will
    // be used for variables later.
    // It will represent a single voter.
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted;  // if true, that person already voted
        address delegate; // person delegated to
        uint vote;   // index of the voted proposal
    }

    // This is a type for a single proposal.
    struct Proposal
    {
        string name;    // Changed from bytes32 to string
        uint voteCount;
    }
    address public chairperson;

    // This declares a state variable that
    // stores a 'Voter' struct for each possible address.
    mapping(address => Voter) public voters;

    // A dynamically-sized array of 'Proposal' structs.
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames) { // Changed bytes32[] to string[]
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
        for (uint i = 0; i < proposalNames.length; i++) {
            proposals.push(Proposal({
```

```

        name: proposalNames[i],
        voteCount: 0
    }));
}
}

/**
 * @dev Give 'voter' the right to vote on this ballot. May only be called by
'chairperson'.
 * @param voter address of voter
 */
function giveRightToVote(address voter) external {
    // If the first argument of `require` evaluates
    // to 'false', execution terminates and all
    // changes to the state and to Ether balances
    // are reverted.
    // This used to consume all gas in old EVM versions, but
    // not anymore.
    // It is often a good idea to use 'require' to check if
    // functions are called correctly.
    // As a second argument, you can also provide an
    // explanation about what went wrong.
    require(
        msg.sender == chairperson,
        "Only chairperson can give right to vote."
    );
    require(
        !voters[voter].voted,
        "The voter already voted."
    );
    require(voters[voter].weight == 0, "Voter already has the right to vote.");
    voters[voter].weight = 1;
}

/**
 * @dev Delegate your vote to the voter 'to'.
 * @param to address to which vote is delegated
 */
function delegate(address to) external {
    // assigns reference
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");

    require(to != msg.sender, "Self-delegation is disallowed.");

    // Forward the delegation as long as
    // 'to' also delegated.
    // In general, such loops are very dangerous,
    // because if they run too long, they might
    // need more gas than is available in a block.
    // In this case, the delegation will not be executed,
    // but in other situations, such loops might
    // cause a contract to get "stuck" completely.

```

```

while (voters[to].delegate != address(0)) {
    to = voters[to].delegate;

    // We found a loop in the delegation, not allowed.
    require(to != msg.sender, "Found loop in delegation.");
}

Voter storage delegate_ = voters[to];

// Voters cannot delegate to accounts that cannot vote.
require(delegate_.weight >= 1);

// Since 'sender' is a reference, this
// modifies 'voters[msg.sender]'.
sender.voted = true;
sender.delegate = to;

if (delegate_.voted) {
    // If the delegate already voted,
    // directly add to the number of votes
    proposals[delegate_.vote].voteCount += sender.weight;
} else {
    // If the delegate did not vote yet,
    // add to her weight.
    delegate_.weight += sender.weight;
}
}

/**
 * @dev Give your vote (including votes delegated to you) to proposal
'proposals[proposal].name'.
 * @param proposal index of proposal in the proposals array
 */
function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");
    sender.voted = true;
    sender.vote = proposal;

    // If 'proposal' is out of the range of the array,
    // this will throw automatically and revert all
    // changes.
    proposals[proposal].voteCount += sender.weight;
}

/**
 * @dev Computes the winning proposal taking all previous votes into account.
 * @return winningProposal_ index of winning proposal in the proposals array
 */
function winningProposal() public view
    returns (uint winningProposal_)
{
    uint winningVoteCount = 0;

```

```

        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

    /**
     * @dev Calls winningProposal() function to get the index of the winner
    contained in the proposals array and then
     * @return winnerName_ the name of the winner
    */
    function winnerName() external view
        returns (string memory winnerName_)
    {
        winnerName_ = proposals[winningProposal()].name;
    }
}

```

Output:-

1)Compile the Ballot.sol Contract

The screenshot displays the Solidity Compiler interface within the Remix IDE. The left sidebar contains the 'SOLIDITY COMPILER' section with options for 'Auto compile' and 'Hide warnings'. Below these are 'Advanced Configurations' and a 'Compile 3_Ballot.sol' button. Further down, there are buttons for 'Compile and Run script', 'Run Remix Analysis', 'Run SolidityScan', 'Publish on IPFS', 'Publish on Swarm', and 'Compilation Details'. The main area shows the 'Ballot' contract with its source code, compiler input, name, metadata, and bytecode. The bottom panel shows the terminal output, which includes a welcome message and instructions on how to use the terminal.

SOLIDITY COMPILER

- ☐ Auto compile
- ☐ Hide warnings
- Advanced Configurations >
- [Compile 3_Ballot.sol](#)
- [Compile and Run script](#)
- CONTRACT
- Ballot (3_Ballot.sol)
- [Run Remix Analysis](#)
- [Run SolidityScan](#)
- [Publish on IPFS](#)
- [Publish on Swarm](#)
- [Compilation Details](#)
- [ABI](#) [Bytecode](#)

Ballot

- COMPILERINPUT [i](#)
- NAME [i](#)
- METADATA [i](#)
 - compiler:
 - language: Solidity
 - output:
 - settings:
 - sources:
 - version: 1
- BYTECODE [i](#)

0 ☐ Listen on all transactions [Filter](#)

Welcome to Remix 1.5.1

Your files are stored in indexedDB, 1.26 KB / 496.46 MB used

You can use this terminal to:

- Check transactions details and start debugging.
- Execute JavaScript scripts:
 - Input a script directly in the command line interface

2) Deploying and running of the contract

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active. The contract 'Ballot - contracts/3_Ballot.sol' is selected. The 'GAS LIMIT' is set to 'Estimated Gas' (3000000). The 'VALUE' is set to '0 Wei'. The 'Deploy' button is highlighted. Below it, the 'At Address' button is also visible. The 'Transactions recorded' section shows one transaction: 'BALLOT AT 0XD91...39138 (MEN)'. The 'Deployed Contracts' section shows the contract 'BALLOT AT 0XD91...39138 (MEN)'. The main editor shows the Solidity code for the 'Ballot' contract. The code includes a constructor, a 'winningProposal()' function, and a 'winnerName()' function. The 'winnerName()' function is highlighted. The 'Explain contract' panel is open, showing the transaction details for the deployment. The transaction hash is '0x2784cb329b4f7c008e564df3556d5bd5cc4c89b8869ba448020d9b281b4b12c5'. The block hash is '0x58619101b728a39a7189b6543b1850f8f86ca377901f4f04695b936ef812a7fe'. The block number is '1'. The contract address is '0xd9145CCE52D386f254917e481e844e9943f39138'.

```
154     }
155   }
156 }
157
158 /**
159  * @dev Calls winningProposal() function to get the index of the winner contained in the proposals array
160  * @return winnerName_ the name of the winner
161  */
162 //Aryan Patankar D20A 38
163 function winnerName() external view {
164     returns (string memory winnerName_)
165 }
166 winnerName_ = proposals[winningProposal()].name;
```

Explain contract

Type the library name to see available commands.
creation of Ballot pending...

✓ [vm] from: 0x583...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...0000 logs: 0 hash: 0x278...b12c5

status	1 Transaction mined and execution succeed
transaction hash	0x2784cb329b4f7c008e564df3556d5bd5cc4c89b8869ba448020d9b281b4b12c5
block hash	0x58619101b728a39a7189b6543b1850f8f86ca377901f4f04695b936ef812a7fe
block number	1
contract address	0xd9145CCE52D386f254917e481e844e9943f39138

3) Loading the proposal candidate's name(string)

The screenshot shows the Remix IDE interface. The 'DEPLOY & RUN TRANSACTIONS' panel is active. The contract 'Ballot - contracts/3_Ballot.sol' is selected. The 'GAS LIMIT' is set to 'Estimated Gas' (3000000). The 'VALUE' is set to '0 Wei'. The 'Deploy' button is highlighted. Below it, the 'At Address' button is also visible. The 'Transactions recorded' section shows one transaction: 'BALLOT AT 0XD91...39138 (MEN)'. The 'Deployed Contracts' section shows the contract 'BALLOT AT 0XD91...39138 (MEN)'. The main editor shows the Solidity code for the 'Ballot' contract. The code includes a constructor, a 'winningProposal()' function, and a 'winnerName()' function. The 'winningProposal()' function is highlighted. The 'Explain contract' panel is open, showing the transaction details for the execution. The transaction hash is '0x2784cb329b4f7c008e564df3556d5bd5cc4c89b8869ba448020d9b281b4b12c5'. The block hash is '0x58619101b728a39a7189b6543b1850f8f86ca377901f4f04695b936ef812a7fe'. The block number is '1'. The contract address is '0xd9145CCE52D386f254917e481e844e9943f39138'.

```
142 /**
143  * @dev Computes the winning proposal taking all previous votes into account.
144  * @return winningProposal_ index of winning proposal in the proposals array
145  */
146 function winningProposal() public view {
147     returns (uint winningProposal_)
148 }
149 {
150     uint winningVoteCount = 0;
151     for (uint p = 0; p < proposals.length; p++) {
152         if (proposals[p].voteCount > winningVoteCount) {
153             winningVoteCount = proposals[p].voteCount;
154             winningProposal_ = p;
155         }
156     }
157 }
```

Explain contract

Type the library name to see available commands.
creation of Ballot pending...

✓ [vm] from: 0x583...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...0000 logs: 0 hash: 0x278...b12c5

status	1 Transaction mined and execution succeed
transaction hash	0x2784cb329b4f7c008e564df3556d5bd5cc4c89b8869ba448020d9b281b4b12c5
block hash	0x58619101b728a39a7189b6543b1850f8f86ca377901f4f04695b936ef812a7fe
block number	1
contract address	0xd9145CCE52D386f254917e481e844e9943f39138

decoded input

```
{
  "string[] proposalNames": [
    "Aryan Patankar",
    "Vaishnal",
    "Niraj"
  ]
}
```

decoded output

```
-
```

logs

```
[]
```

raw logs

```
[]
```

4) Viewing the details of the Proposal Candidate

▼ BALLOT AT 0XD91...39138 (ME)   

Balance: 0 ETH

delegate

address to

▼

giveRightToVote

address voter

▼

vote

uint256 proposal

▼

chairperson

0: address: 0x5B38Da6a701c568545
dCfcB03FcB875f56beddC4

proposals

◀ proposals - call

▼

0: string: name Aryan Patankar

1: uint256: voteCount 0

voters

address

▼

winnerName

winningProposal

Low level interactions 

CALLDATA

Transact

5) Giving the right to an account other than and by the chairman

Deploy & Run Transactions

Deploy: [Aryan Patankar, "Vaishna", "Niraj"]

At Address: Load contract from Address

Transactions recorded: 2

Deployed Contracts

BALLOT AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

delegate: address to

giveRightToVote: 0xAb8483F64d9C6d1EcF9b849Ae677dD33

vote: uint256 proposal

chairperson

0: address: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4

proposals: 0

0: string: name Aryan Patankar

1: uint256: voteCount 0

voters: address

Solidity Compile Details

```

142  /**
143   * @dev Computes the winning proposal taking all previous votes into account.
144   * @return winningProposal_ index of winning proposal in the proposals array
145   */
146  function winningProposal() public view infinite gas
147      returns (uint winningProposal_)
148  {

```

Explain contract

status: 1 Transaction mined and execution succeed

transaction hash: 0x0c939ce8f8b5336f86beda71feaa691ecb80c8cc821958f8bcbef88556d6

block hash: 0x57dc15c9fa967d05463781d80e27e3e069666fd1613041285d6674a59fc85c25

block number: 2

from: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4

to: Ballot.giveRightToVote(address) 0xd9145CCE52D386f254917e481e844e9943f39138

transaction cost: 48642 gas

execution cost: 27210 gas

output: 0x

decoded input: { "address voter": "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2" }

6) Selecting the account which was given the right to vote and then writing the proposal candidate's index to vote.

Deploy & Run Transactions

Deploy: [Aryan Patankar, "Vaishna", "Niraj"]

At Address: Load contract from Address

Transactions recorded: 2

Deployed Contracts

BALLOT AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

delegate: address to

giveRightToVote: 0xAb8483F64d9C6d1EcF9b849Ae677dD33

vote: 0

chairperson

0: address: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4

proposals: 0

0: string: name Aryan Patankar

1: uint256: voteCount 0

voters: address

winnerName

winningProposal

Low level interactions

CALLDATA

Transact

Solidity Compile Details

```

142  /**
143   * @dev Computes the winning proposal taking all previous votes into account.
144   * @return winningProposal_ index of winning proposal in the proposals array
145   */
146  function winningProposal() public view infinite gas
147      returns (uint winningProposal_)
148  {

```

Explain contract

raw logs: []

transact to Ballot.vote pending ...

[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...0000 logs: 0 hash: 0xc66...d2510

status: 1 Transaction mined and execution succeed

transaction hash: 0xc66880f36854abf57f4f875a36b5bfae3fa7c2716655cd6470628a9a2d2510

block hash: 0x60889ae631b336a186dd93ba5886887857f2064e5cb2c6957609df9f28f98db

block number: 3

from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2

to: Ballot.vote(uint256) 0xd9145CCE52D386f254917e481e844e9943f39138

transaction cost: 72984 gas

7) We can view the Voter's Information

The screenshot displays a web application interface for viewing voter information and transaction details. On the left, under the 'voters' tab, a list of voter data is shown:

- 0: uint256: weight 1
- 1: bool: voted true
- 2: address: delegate 0x00
- 3: uint256: vote 0

Below this list are buttons for 'winnerName' and 'winningProposal'. A 'Low level interactions' section shows a 'CALLDATA' input field and a 'Transact' button. On the right, a transaction status is shown as 'transact to Ballot.vote pending ...'. Below this, a green checkmark indicates the transaction was successful. The transaction details are as follows:

Field	Value
status	1 Transaction mined and execution succeed
transaction hash	0xc66880f36854abf57f4f875a36b5bfafe3fa7c2716655cd6470628a9a2d2510
block hash	0x60889ae631b336a186dd93ba5086887857f2064e5cb2c66957609df9f28f98db
block number	3
from	0xab8483f64d9c6d1ecf9b849ae677d03315835cb2
to	Ballot.vote(uint256) 0xd9145CCE52D386f254917e481e844e9943f39138
transaction cost	72984 gas
execution cost	51792 gas
output	0x

8) Selecting the account of the delegator and then writing the address of the voter to be delegated to.

The screenshot shows a web application interface for deploying and running transactions. The left sidebar contains the following sections:

- ENVIRONMENT:** Remix VM (Cancun)
- ACCOUNT:** 0xab8...35cb2 (99.999999999999748988 ETH)
- GAS LIMIT:** Estimated Gas (3000000)
- VALUE:** 0 Wei
- CONTRACT:** Ballot - contracts/3_Ballot.sol
- Deploy:** [Aryan Patankar, "Vaishnal", "Niraj"]
- At Address:** Load contract from Address

The main area displays the Solidity code for the `winningProposal()` function:

```
142 /**
143  * @dev Computes the winning proposal taking all previous votes into account.
144  * @return winningProposal_index of winning proposal in the proposals array
145  */
146 function winningProposal() public view { infinite gas
```

Below the code, the 'Explain contract' tab is active, showing the transaction details:

Field	Value
block number	7
from	0x40209938c481177ec7e8f571ceCaEBA9e22C02db
to	Ballot.delegate(address) 0xd9145CCE52D386f254917e481e844e9943f39138
transaction cost	61199 gas
execution cost	39767 gas
output	0x
decoded input	{ "address to": "0xab8483f64d9c6d1ecf9b849ae677d03315835cb2" }
decoded output	()
logs	[]
raw logs	[]

At the bottom, it shows 'transact to Ballot.vote pending ...' and 'Transactions recorded 10'.

DEPLOY & RUN TRANSACTIONS

CONTRACT

Ballot - contracts/3_Ballot.sol

evm version: osaka

Deploy [Aryan Patankar, "Vaishnal", "Niraj"]

At Address Load contract from Address

Transactions recorded 10

Deployed Contracts 1

BALLOT AT 0xD91...39138 (MEMORY)

Balances: 0 ETH

delegate 0xab8483f64d9c6d1ecf9b849ae677d033

giveRightToVote 0x48209938c481177ec7e8f571ceCaE8A9e2

vote < vote - transact (not payable)

chairperson

142 /**

143 * @dev Computes the winning proposal taking all previous votes into account.

144 * @return winningProposal_index of winning proposal in the proposals array

145 */

146 function winningProposal() public view infinite gas

Explain contract

block number 7

from 0x4B209938c481177ec7E8f571ceCaE8A9e22C82db

to Ballot.delegate(address) 0xd9145CCE52D386f254917e481eB44e9943F39138

transaction cost 61199 gas

execution cost 39767 gas

output 0x

decoded input

{ "address to": "0xab8483f64d9c6d1ecf9b849ae677d03315835cb2"

decoded output {}

logs []

raw logs []

9) Proposal Candidate's Information before giving the Delegate's vote

proposals 0

0: string: name Aryan Patankar

1: uint256: voteCount 3

10) In this final screenshot we can see that after the delegation's vote the weight of one vote of the one voter who was delegated is the number of people who delegated the voter as their voter (Default weight of any voter is 1)

The screenshot displays the Remix IDE interface. On the left, the 'LOCAL & RUN TRANSACTIONS' panel shows a transaction with the following details:

- vote**: 0
- chairperson**: 0x5838Da6a701c568545dCfc803FcB875f56beddC4
- proposals**: 0
- voters**: 0xAb8483F6d9C6d1EcF9b849Ae677dD33
- winnerName**: 0x string: winnerName_ Aryan Patankar
- winningProposal**: winningProposal - call

The main editor shows the Solidity code for the `winningProposal()` function:

```
142 /**  
143  * @dev Computes the winning proposal taking all previous votes into account.  
144  * @return winningProposal_index of winning proposal in the proposals array  
145  */  
146 function winningProposal() public view {  
    // ...  
}
```

The right panel shows the 'Explain contract' view, displaying the decoded input and output of the transaction. The decoded output shows the winning proposal index as 0.

The bottom panel shows the 'CALLDATA' section, indicating the transaction hash and the function being called.

Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.